

The Takeaway Card

What to keep after today

You built something today. This page is what makes the next one easier — the reflexes on this side, the judgment on the back. Keep it next to your keyboard for the first month.

The loop, every time

Describe → Plan → Build → Run → Debug. That is the whole skill — it does not get harder on bigger jobs.

- **Describe** the goal, not the method. Brief Claude the way you would brief a sharp new colleague: what you want, what you will judge it by, what it must not do.
- **Plan first.** On anything non-trivial, ask for a plan before code — or use Plan Mode. Read it. Push back. Nothing has touched your files yet.
- **Build and run**, then actually look at the result.
- **Debug** by pasting the error back. Starting over is the only real mistake.

When it breaks — four moves that cover most of it

1. **Paste the exact error back** and say "this happened." Claude diagnoses far better than it self-checks. This is the highest-value habit on the page.
2. **Check the address bar.** A page that reads a file needs a web server. `localhost` is your running app; a long `/Users/...` path is only a preview, and anything it loads will fail. Say *"start a web server in this folder and open the localhost page."*
3. **Ask for less.** If Claude scaffolds a framework or installs things you did not ask for: *"simpler please — one file, no installs, plain HTML/CSS/JS."*
4. **Undo it.** *"Undo that." "Put it back how it was."* Nothing you build is precious, so build boldly.

Three questions before you build anything

- Does it only affect **my own work**?
- Does it only read data I **already have permission to see**?
- Can I **throw it away** if it breaks?

Three yeses → 🟢 **Safe.** Build it. That covers most of what you will actually want.

Writes to shared systems, others rely on it, or it runs unattended → 🟡 **Review.** Talk to someone before it becomes load-bearing.

Production, customer-facing, or regulated data → 🔴 **Stop.** That is engineering's.

If you cannot tell which zone you are in, you are in the middle one. Ask.

The judgment layer

The fundamentals get you a working tool. These three habits keep it working.

1. Test it like you would check a new hire's work

You own the output, even though Claude typed it. Thirty seconds each:

- **Golden example.** Feed it one input where you already know the answer. If it reports 199 rows in a file you know has 200, stop there.
- **Try to break it once.** An empty file, a strange date, a name with a comma. How something fails tells you more than watching it succeed.
- **Ask Claude to test its own work.** *"Write a quick check that proves this handles the file correctly, then run it."* It verifies better than you would expect — but only when asked.

2. Model selection, without the leaderboard

You do not need to follow model releases. Two questions cover it:

- **Mechanical, or judgment-heavy?** Renaming files or reformatting a spreadsheet: the default model is fine. Reasoning about *why* numbers disagree, or work you will base a decision on: reach for the most capable model.
- **Did the first answer feel shallow?** Escalate the model, or ask for a plan before it tries again. Nine times out of ten the fix is a better prompt, not a better model.

3. The thousand lines you do not understand

Claude will sometimes hand you more code than you can read. Do not skim it and hope:

1. **Ask for the tour.** *"Explain what this does in plain English, section by section, and list anything risky."*
2. **Ask for less.** *"Rewrite this as simply as possible — fewer features, fewer files, standard library only."* Most have a shorter version you can follow.
3. **Ask for the proof.** *"Add a check I can run that shows it works on this example."*
4. **Run the zone test again.** Code you do not understand can still be Safe — your data, throwaway, nobody depending on it. Once someone else relies on it, it is a Review conversation: bring the plain-English tour to it.

The rule underneath all three: you do not need to write code to be responsible for it — and Claude will help you carry that responsibility, if you ask.

Where to go from here

- **Build the boring thing first** — whatever you already do by hand every week.
- **Stuck for ten minutes?** Ask a person, same as today.
- **Your support channel** is where prompts and wins compound. Yours will unblock someone.